



FLIGHT BOOKING SYSTEM

FBS

Final Project Report Presentation
Data Process and Software Modelling

Christian Cantos | 23188023

SUMMARY

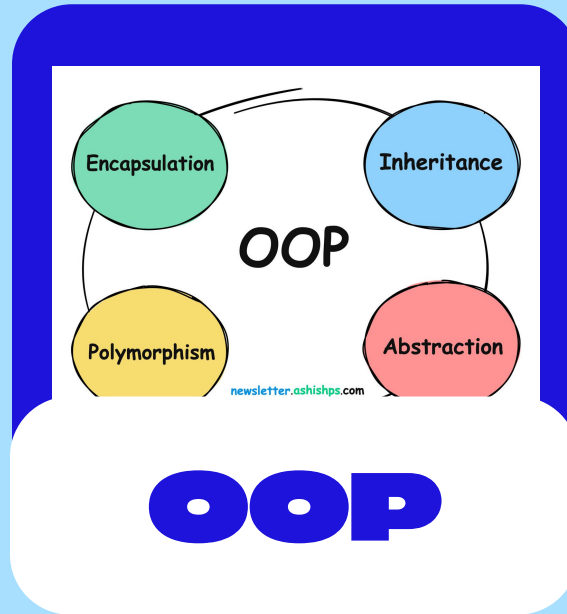
- Project Overview
- System Architecture
- Refined UML Class Diagram
- Key Improvements
- Design Patterns
- Sequence Diagram
- Activity Diagram
- Challenges & Lessons Learned
- Future Improvements



PROJECT OVERVIEW

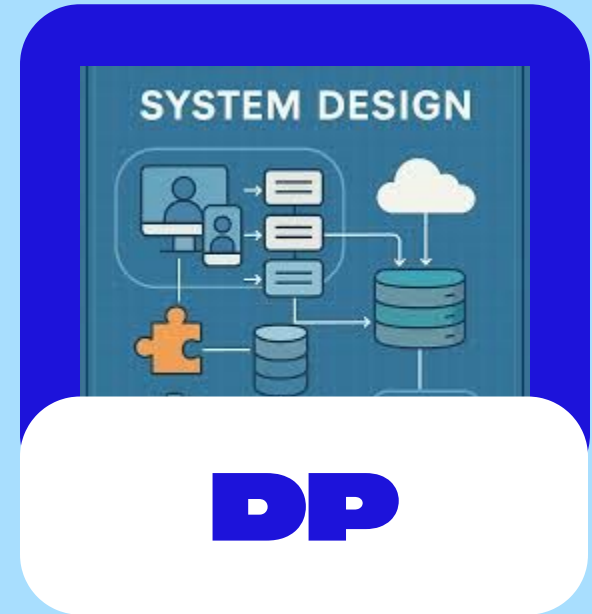


Developed in C#



OOP

Applied Objected
Oriented Principles



DP

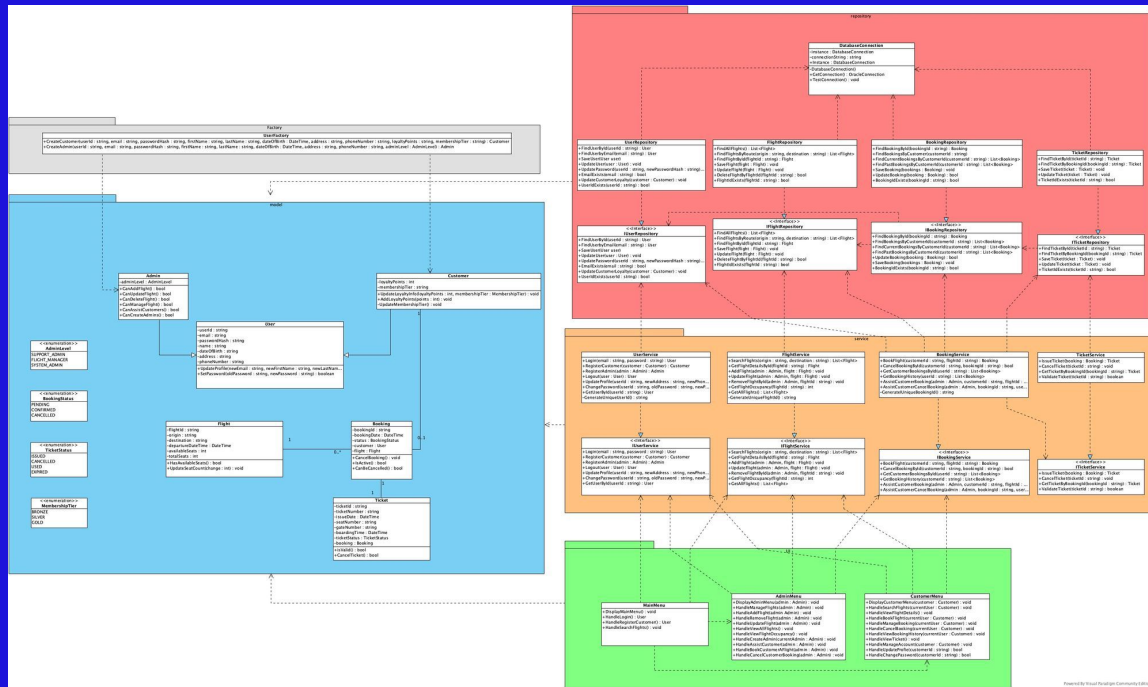
Implementation of
System Design

SYSTEM ARCHITECTURE

- Layered Architecture
- Learned from previous project
- Maintainability
- Separation of Concerns
- Reduces coupling



REFINED UML CLASS DIAGRAM



- **Introduced Inheritance**
- **Specialised Subclasses**
 - **Customer**
 - **Admin**
- **Eliminated extensive role-checking**
- **Improved object oriented structure through abstraction and polymorphism.**
- **Role Checking**
- **New Ticket Entity**
- **Introduced Interfaces**
- **Improve separation of concerns**
- **Increase system maintainability, scalability, and extensibility**
- **Better reflects real-world airling booking operations**

KEY IMPROVEMENTS

Initial Design

- Single User class with Role Enumeration
- Extensive role-checking
- No Ticket Entity
- Basic domain model
- Direct object creation using constructors
- No centralised connection management
- Less specialised responsibilities
- Limited extensibility for new user types
- Lower cohesion in user management
- Higher coupling between components

Refined Design

- Abstract user superclass with Customer and Admin subclasses
- Uses inheritance and polymorphism
- Added Ticket entity for ticket management
- More complete airline reservation model
- Factory Method Pattern through UserFactory
- Singleton Pattern through DatabaseConnection
- Clear separation of responsibilities
- Easier to add new user types and features
- Improved cohesion through specialised subclasses
- Reduced coupling through interfaces and layered architecture

DESIGN PATTERNS

- Singleton Pattern
- Factory Method



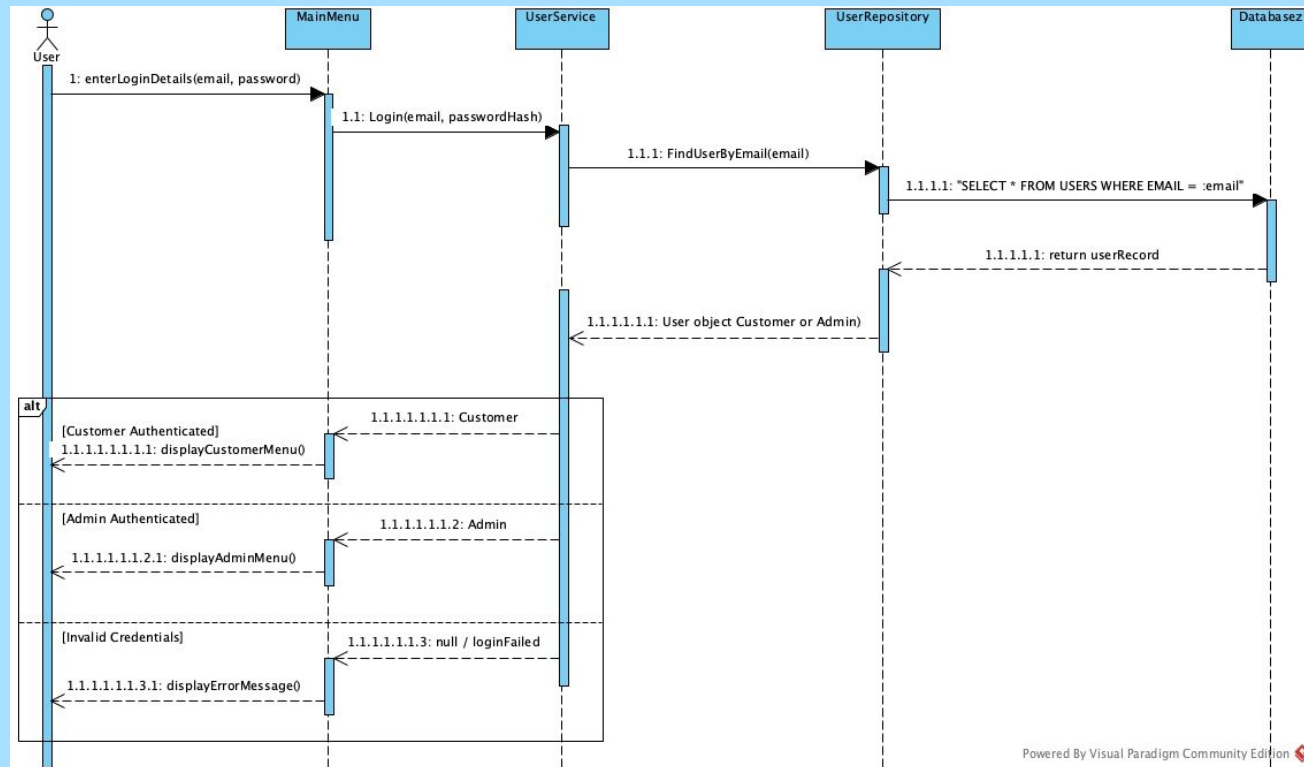
C# Implementation

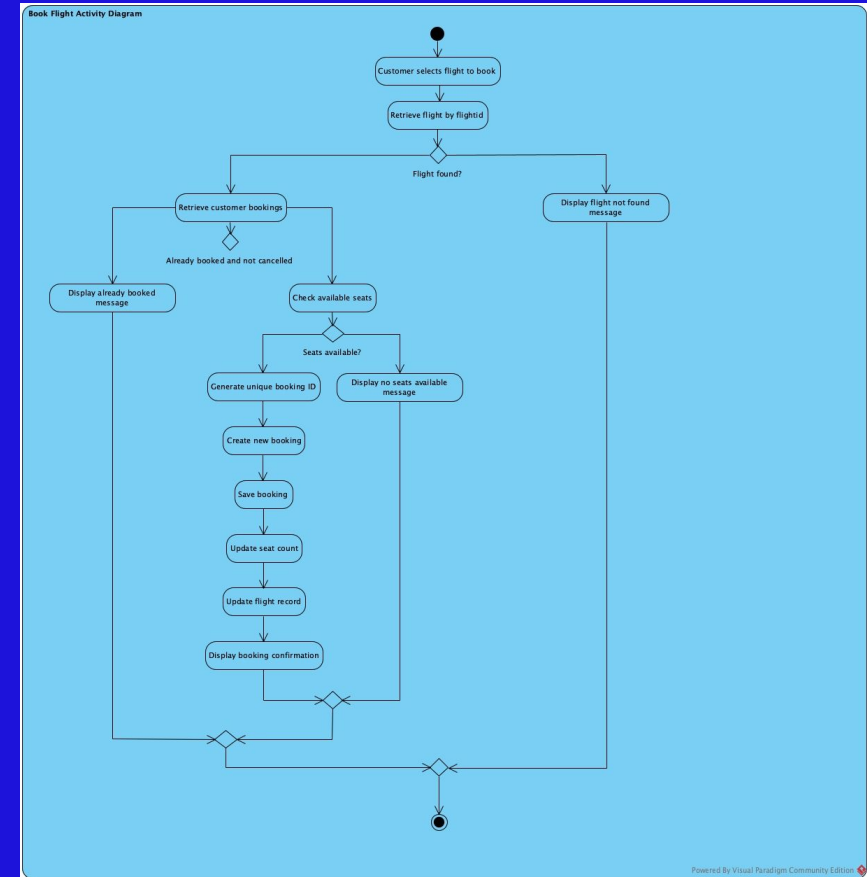
```
FlightBookingSystem.Factory > UserFactory > CreateAdmin(string, string, string, string, string, Date
1 using FlightBookingSystem.Model;
2
3 namespace FlightBookingSystem.Factory
4 {
5     0 references
6     public static class UserFactory
7     {
8         0 references
9         public static Customer CreateCustomer(
10             string userId,
11             string email,
12             string passwordHash,
13             string firstName,
14             string lastName,
15             DateTime dateOfBirth,
16             string address,
17             string phoneNumber,
18             int loyaltyPoints,
19             string membershipTier)
20         {
21             return new Customer(userId, email, passwordHash, firstName, lastName,
22                 dateOfBirth, address, phoneNumber);
23         }
24
25         0 references
26         public static Admin CreateAdmin(
27             string userId,
28             string email,
29             string passwordHash,
30             string firstName,
31             string lastName,
32             DateTime dateOfBirth,
33             string address,
34             string phoneNumber,
35             AdminLevel adminLevel)
36         {
37             return new Admin(userId, email, passwordHash, firstName, lastName,
38                 dateOfBirth, address, phoneNumber, adminLevel);
39         }
40     }
41 }
```

```
Repository > DatabaseConnection.cs > FlightBookingSystem.Repository > DatabaseConnection > GetConne
1 using Oracle.ManagedDataAccess.Client;
2
3 namespace FlightBookingSystem.Repository
4 {
5     31 references
6     public sealed class DatabaseConnection
7     {
8         3 references
9         private static DatabaseConnection? instance; // Singleton Pattern
10
11         2 references
12         private readonly string connectionString;
13
14         // Private constructor prevents external instantiation
15         private DatabaseConnection()
16         {
17             connectionString = "User Id=system;Password=1234;Data Source=localhost:1521/xes";
18         }
19
20         // Global access point
21         27 references
22         public static DatabaseConnection Instance
23         {
24             get
25             {
26                 if(instance == null)
27                 {
28                     instance = new DatabaseConnection();
29                 }
30                 return instance;
31             }
32         }
33
34         28 references
35         public OracleConnection GetConnection()
36         {
37             return new OracleConnection(connectionString);
38         }
39
40         0 references
41         public void TestConnection()
42         {
43             using OracleConnection conn = GetConnection();
44             conn.Open();
45
46             Console.WriteLine("Database connected successfully!");
47             Console.WriteLine($"Data Source: {conn.DataSource}");
48
49             using OracleCommand cmd = new OracleCommand("SELECT USER FROM dual", conn);
50             string currentUser = cmd.ExecuteScalar()?.ToString();
51
52             Console.WriteLine($"Connected as: {currentUser}");
53         }
54     }
55 }
```


SEQUENCE DIAGRAM

Login Sequence Diagram





Book Flight Activity Diagram



CHALLENGES & LESSON LEARNED

- Role Enum to Enheritance
- Oracle Database Integration
- Keeping UML aligned with code
- Importance of System Design

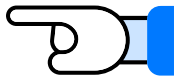
FUTURE IMPROVEMENTS



**PAYMENT
PROCESSING**



**ADDITIONAL
USER TYPES**

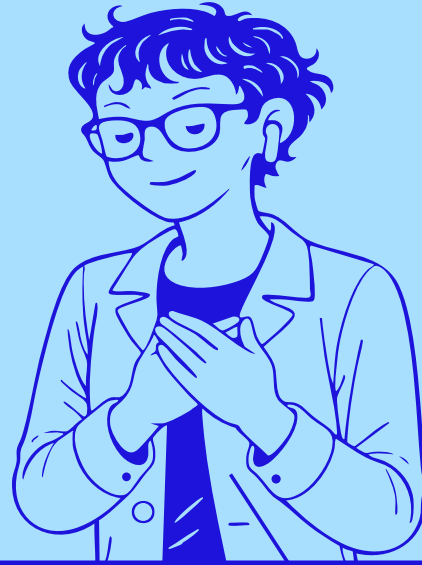


**SEAT
SELECTION**



**BETTER
INTERFACE**





THANK YOU!